

Q) GitHub Repository Management, Stash, Tagging and Log Analysis

Create a GitHub repository named **Lab_Git_<prn>** using your GitHub account. Initialize a local Git repository named **Lab_Git** and create a file named **Exam_details.txt** containing your PRN number. Commit the file and connect the local repository to the GitHub repository using your GitHub username and Personal Access Token (PAT). Push the repository to GitHub and verify that the files are available remotely.

Modify the file by adding your full name and create a second commit. Make additional uncommitted changes containing a welcome message and save these changes using Git Stash. Create a new branch named **feature**, switch to it, and verify that the stashed changes are not present. Restore the stashed changes in the feature branch and commit them.

Create another commit by adding the current date and time to the file. Create a lightweight tag named **v1.0** on the current commit and an annotated tag named **release-v1.1** with an appropriate description. Display all available tags and examine the information stored in the annotated tag.

Use Git log commands to display the commit history in different formats, including commit IDs, author information, commit dates, and branch relationships. Identify the commit associated with each tag. Check out the repository using the **v1.0** tag and verify the file contents. Return to the latest branch and confirm that all commits and tags remain intact both locally and on GitHub.

Finally, push all commits and tags to GitHub and verify that the repository history, branches, stashes, and tags have been managed correctly throughout the exercise.

—

Step 1: Create a Local Git Repository

We create a local Git repository named **Lab_Git** to track file changes before connecting it to GitHub.

Command

```
mkdir Lab_Git && cd Lab_Git
```

```
git init
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git
$ git init
Initialized empty Git repository in D:/learn/git/stashTagLog/Lab_Git/.git/
```

Step 2: Create Exam Details File and First Commit

We create a file containing the PRN number and save it as the first version of the project.

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "Initial commit with PRN"
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git
$ git init
Initialized empty Git repository in D:/learn/git/stashTagLog/Lab_Git/.git/

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ vim Exam_details_60.txt

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ cat Exam_details_60.txt
first commit

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git add .
warning: in the working copy of 'Exam_details_60.txt', LF
will be replaced by CRLF the next time Git touches it

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git add .
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git commit -m "Initial commit with PRN"
Author identity unknown

*** Please tell me who you are.

Run

    git config --global user.email "you@example.com"
    git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository
.

fatal: unable to auto-detect email address (got 'Lenovo@theMist.(none)')
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git config --global user.name "the7mist"

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git config --global user.email "asombale@gmail.com"

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git config --global user.name
the7mist

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git config --global user.email
asombale@gmail.com
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git commit -m "Initial commit with PRN"
[main (root-commit) 3df2272] Initial commit with PRN
 1 file changed, 1 insertion(+)
 create mode 100644 Exam_details_60.txt

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git push origin main --force
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 242 bytes | 242.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/THE7MIST/Lab_Git_260240123060.git
 * [new branch]      main -> main
```

File Content

First commit

Explanation of attributes

- `git add .` → Stages all files.
- `git commit` → Creates a repository snapshot.
- `-m` → Commit message.

Step 3: Connect Local Repository to GitHub

We link the local repository to the GitHub repository named **Lab_Git_**.

The screenshot shows the GitHub interface for a repository named 'Lab_Git_260240123060' by user 'THE7MIST'. The repository is public. The page includes navigation tabs for Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, Insights, and Settings. Below the repository name, there are buttons for Pin, Watch (0), Fork (0), and Star (0). The main content area has two cards: 'Start coding with Codespaces' and 'Add collaborators to this repository'. Below these is a 'Quick setup' section with a button to 'Set up in Desktop' or 'HTTPS' or 'SSH' and a text box containing the repository URL: 'https://github.com/THE7MIST/Lab_Git_260240123060.git'. Below the quick setup is a section titled '...or create a new repository on the command line' with a code block containing the following commands:

```
echo "# Lab_Git_260240123060" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/THE7MIST/Lab_Git_260240123060.git
git push -u origin main
```

Below this is another section titled '...or push an existing repository from the command line' with a code block containing the following commands:

```
git remote add origin https://github.com/THE7MIST/Lab_Git_260240123060.git
git branch -M main
git push -u origin main
```

Command

git remote add origin https://github.com/<USERNAME>/Lab_Git_<PRN>.git

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$git remote add origin https://github.com/THE7MIST/Lab_Git_260240123060.git

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git branch -M main
```

Explanation of attributes

- **remote add** → Creates a remote connection.
- **origin** → Default remote repository name.
- **URL** → GitHub repository address.

Step 4: Push Repository to GitHub

What are we doing and why?

We upload the local repository to GitHub for remote storage and collaboration.

Command

git branch -M main

git push -u origin main

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git push origin main --force
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 242 bytes | 242.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/THE7MIST/Lab_Git_260240123060.git
 * [new branch]      main -> main
```

Explanation of attributes

- **branch -M main** → Renames current branch to main.
- **push** → Uploads commits.
- **-u** → Sets upstream tracking branch.
- **origin main** → Pushes to main branch on GitHub.

Step 5: Add Name and Create Second Commit

We modify the file by adding the student's name and save it as another version.

Command

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "Added student name"
```

File Content

PRN: <PRN_NUMBER>

Name: <FULL_NAME>

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ vim Exam_details_60.txt

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ cat Exam_details_60.txt
first commit
PRN: 60
Name: JJJJ

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git add .
warning: in the working copy of 'Exam_details_60.txt', LF will be
replaced by CRLF the next time Git touches it

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git commit -m "Added student name"
[main 435fa7a] Added student name
1 file changed, 2 insertions(+)
```

Explanation of attributes

- Creates a second commit in repository history.

Step 6: Create Uncommitted Changes

We add a welcome message but intentionally do not commit it to demonstrate Git Stash.

Command

```
vim Exam_details_<PRN>.txt
```

Added Content

Welcome to GitHub Repository Management Lab

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ vim Exam_details_60.txt

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ cat Exam_details_60.txt
first commit
PRN: 60
Name: JJJJ
Welcome to GitHub Repository Management Lab
```

Step 7: Save Changes Using Git Stash

Git Stash temporarily stores uncommitted changes without creating a commit.

Command

```
git stash
```

Explanation of attributes

- **stash** → Saves current working directory changes temporarily.

```

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Exam_details_60.txt

no changes added to commit (use "git add" and/or "git commit -a")
)

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git stash
warning: in the working copy of 'Exam_details_60.txt', LF will be
replaced by CRLF the next time Git touches it
Saved working directory and index state WIP on main: 435fa7a Add
ed student name

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git stash
No local changes to save

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git status
On branch main
nothing to commit, working tree clean

```

Step 8: Create Feature Branch

What are we doing and why?

We create a new branch for feature development.

Command

git checkout -b feature

```

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (main)
$ git checkout -b feature
Switched to a new branch 'feature'

```

Explanation of attributes

- **checkout** → Switches branch.
- **-b** → Creates branch.
- **feature** → Branch name.

Step 9: Verify Stashed Changes Are Not Present

We confirm that stashed changes are removed from the working directory.

Command

```
cat Exam_details_<PRN>.txt
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ cat Exam_details_60.txt
first commit
PRN: 60
Name: JJJJ
```

Explanation of attributes

- Displays current file content.

Step 10: Restore Stashed Changes

What are we doing and why?

We apply the stashed changes inside the feature branch.

Command

```
git stash pop
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git stash pop
On branch feature
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Exam_details_60.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (98f1112ed8fef38bd98a20af6ce56ebafcbef1d7)
```

Explanation of attributes

- `stash pop` → Restores and removes stash entry.

Step 11: Commit Restored Changes

What are we doing and why?

We save the restored welcome message permanently.

Command

```
git add .
```

```
git commit -m "Added welcome message"
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git commit -m "Added welcome message"
[feature 1096968] Added welcome message
1 file changed, 1 insertion(+)
```

Explanation of attributes

- Creates another commit on feature branch.

Step 12: Add Current Date and Time

What are we doing and why?

We create another version before tagging.

Command

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "Added current date and time"
```

Added Content

Date: <CURRENT_DATE>

Time: <CURRENT_TIME>

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ vim Exam_details_60.txt

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ cat Exam_details_60.txt
first commit
PRN: 60
Name: JJJJ
Welcome to GitHub Repository Management Lab
Date: 07/06/2026
Time: 13:09:56

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git commit -m "Added current date and time"
[feature 71544ab] Added current date and time
1 file changed, 2 insertions(+)
```

Step 13: Create Lightweight Tag

We mark the current commit as version 1.0.

Command

git tag v1.0

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git tag v1.0
```

Explanation of attributes

- **tag** → Creates a Git tag.
- **v1.0** → Lightweight tag name.

Step 14: Create Annotated Tag

Annotated tags contain additional metadata such as author, date, and description.

Command

```
git tag -a release-v1.1 -m "Stable release version 1.1"
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git tag -a release-v1.1 -m "Stable release version 1.1"
```

Explanation of attributes

- `-a` → Creates annotated tag.
- `release-v1.1` → Tag name.
- `-m` → Tag description.

Step 15: Display All Tags

We verify all created tags.

Command

```
git tag
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git tag
release-v1.1
v1.0
```

Explanation of attributes

- Lists all available tags.

Step 16: Examine Annotated Tag Information

We inspect metadata stored in the annotated tag.

Command

```
git show release-v1.1
```

```

commit 71544abf0d6ead75683f13a45d513237d4691e6e (HEAD -> feature, tag: v1.0, tag: release-v1.1)
Author: the7mist <asombale@gmail.com>
Date: Sun Jun 7 13:09:38 2026 +0530

    Added current date and time

diff --git a/Exam_details_60.txt b/Exam_details_60.txt
index 75b1fdc..6733834 100644
--- a/Exam_details_60.txt
+++ b/Exam_details_60.txt
@@ -2,3 +2,5 @@ first commit
 PRN: 60
 Name: JJJJ
 Welcome to GitHub Repository Management Lab
+Date: 07/06/2026
+Time: 13:09:56

```

Explanation of attributes

- **show** → Displays tag and commit details.

Step 17: Display Commit History in Different Formats

We analyze repository history using various log views.

Command

git log

git log --oneline

```

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git log
commit 71544abf0d6ead75683f13a45d513237d4691e6e (HEAD -> feature, tag: v1.0, tag: release-v1.1)
Author: the7mist <asombale@gmail.com>
Date: Sun Jun 7 13:09:38 2026 +0530

    Added current date and time

commit 1096968cbabd1cbc853c8b965ea7e1307be544c9
Author: the7mist <asombale@gmail.com>
Date: Sun Jun 7 13:07:46 2026 +0530

    Added welcome message

commit 435fa7a4012c717b04ae146c8af941cd5153f069 (main)
Author: the7mist <asombale@gmail.com>
Date: Sun Jun 7 13:02:25 2026 +0530

    Added student name

commit 3df2272df553e03c2b3f134fab06dea295f4645a (origin/main)
Author: the7mist <asombale@gmail.com>
Date: Sun Jun 7 12:56:10 2026 +0530

    Initial commit with PRN

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git log --oneline
71544ab (HEAD -> feature, tag: v1.0, tag: release-v1.1) Added current date and time
1096968 Added welcome message
435fa7a (main) Added student name
3df2272 (origin/main) Initial commit with PRN

```

git log --graph --all

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git log --graph
* commit 71544abf0d6ead75683f13a45d513237d4691e6e (HEAD -> feature, tag: v1.0, tag: release-v1.1)
  Author: the7mist <asombale@gmail.com>
  Date: Sun Jun 7 13:09:38 2026 +0530

    Added current date and time

* commit 1096968cbabd1cbc853c8b965ea7e1307be544c9
  Author: the7mist <asombale@gmail.com>
  Date: Sun Jun 7 13:07:46 2026 +0530

    Added welcome message

* commit 435fa7a4012c717b04ae146c8af941cd5153f069 (main)
  Author: the7mist <asombale@gmail.com>
  Date: Sun Jun 7 13:02:25 2026 +0530

    Added student name

* commit 3df2272df553e03c2b3f134fab06dea295f4645a (origin/main)
  Author: the7mist <asombale@gmail.com>
  Date: Sun Jun 7 12:56:10 2026 +0530

    Initial commit with PRN

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git log --graph --oneline
* 71544ab (HEAD -> feature, tag: v1.0, tag: release-v1.1) Added current date and time
* 1096968 Added welcome message
* 435fa7a (main) Added student name
* 3df2272 (origin/main) Initial commit with PRN
```

git log --decorate

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git log --decorate
commit 71544abf0d6ead75683f13a45d513237d4691e6e (HEAD -> feature, tag: v1.0, tag: release-v1.1)
Author: the7mist <asombale@gmail.com>
Date: Sun Jun 7 13:09:38 2026 +0530

    Added current date and time

commit 1096968cbabd1cbc853c8b965ea7e1307be544c9
Author: the7mist <asombale@gmail.com>
Date: Sun Jun 7 13:07:46 2026 +0530

    Added welcome message

commit 435fa7a4012c717b04ae146c8af941cd5153f069 (main)
Author: the7mist <asombale@gmail.com>
Date: Sun Jun 7 13:02:25 2026 +0530

    Added student name

commit 3df2272df553e03c2b3f134fab06dea295f4645a (origin/main)
Author: the7mist <asombale@gmail.com>
Date: Sun Jun 7 12:56:10 2026 +0530

    Initial commit with PRN
```

Explanation of attributes

- **--oneline** → Compact view.
- **--graph** → Graphical branch view.
- **--all** → Includes all branches.
- **--decorate** → Shows tags and branch names.

Step 18: Identify Commits Associated with Tags

We determine which commit each tag references.

Command

```
git show v1.0
```

```
git show release-v1.1
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git show release-v1.1
tag release-v1.1
Tagger: the7mist <asombale@gmail.com>
Date: Sun Jun 7 13:12:09 2026 +0530

Stable release version 1.1

commit 71544abf0d6ead75683f13a45d513237d4691e6e (HEAD -> feature, tag: v1.0, tag: release-v1.1)
Author: the7mist <asombale@gmail.com>
Date: Sun Jun 7 13:09:38 2026 +0530

    Added current date and time

diff --git a/Exam_details_60.txt b/Exam_details_60.txt
index 75b1fdc..6733834 100644
--- a/Exam_details_60.txt
+++ b/Exam_details_60.txt
@@ -2,3 +2,5 @@ first commit
 PRN: 60
 Name: JJJJ
 Welcome to GitHub Repository Management Lab
+Date: 07/06/2026
+Time: 13:09:56
```

Explanation of attributes

- Displays commit information linked to tags.

Step 19: Checkout Using Tag v1.0

What are we doing and why?

We move repository state to the tagged version.

Command

```
git checkout v1.0
```

- Checks out the commit referenced by tag v1.0.

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git checkout v1.0
Note: switching to 'v1.0'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 71544ab Added current date and time
```

Step 20: Verify File Content

What are we doing and why?

We confirm the file reflects the tagged version.

Command

```
cat Exam_details_<PRN>.txt
```

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git ((release-v1.1))
$ cat Exam_details_60.txt
first commit
PRN: 60
Name: JJJJ
Welcome to GitHub Repository Management Lab
Date: 07/06/2026
Time: 13:09:56
```


Explanation of attributes

- Displays file content.

Step 21: Return to Latest Branch

We switch back to the feature branch and continue normal development.

Command

git checkout feature

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git ((release-v1.1))
$ git checkout feature
Switched to branch 'feature'
```

Explanation of attributes

- Returns to latest working branch.

Step 22: Push All Commits and Tags to GitHub

What are we doing and why?

We synchronize all local changes, branches, and tags with GitHub.

git push origin feature

git push origin --tags

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git push origin feature
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 839 bytes | 419.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
remote:
remote: Create a pull request for 'feature' on GitHub by visiting:
remote:   https://github.com/THE7MIST/Lab_Git_260240123060/pull/new/feature
remote:
To https://github.com/THE7MIST/Lab_Git_260240123060.git
 * [new branch]      feature -> feature

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git push origin --tags
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 172 bytes | 172.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/THE7MIST/Lab_Git_260240123060.git
 * [new tag]         release-v1.1 -> release-v1.1
 * [new tag]         v1.0 -> v1.0
```

Explanation of attributes

- `origin` → Remote repository.
- `feature` → Branch to push.
- `--tags` → Pushes all tags.

Step 23: Verify Repository Status

We confirm successful management of commits, tags, branches, and stash operations.

Command

git branch

git tag

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git branch
* feature
  main

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git tag
release-v1.1
v1.0
```

git stash list

git log --graph --oneline --all

```
Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git stash list

Lenovo@theMist MINGW64 /d/learn/git/stashTagLog/Lab_Git (feature)
$ git log --graph --oneline --all
* 71544ab (HEAD -> feature, tag: v1.0, tag: release-v1.1, origin/feature) Added current date and time
* 1096968 Added welcome message
* 435fa7a (main) Added student name
* 3df2272 (origin/main) Initial commit with PRN
```

Explanation of attributes

- `branch` → Lists branches.
- `tag` → Lists tags.
- `stash list` → Shows available stashes.
- `log --graph --all` → Displays complete repository history.

Memory Sheet (Command Flow)

```
mkdir Lab_Git
```

```
cd Lab_Git
```

```
git init
```

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "Initial commit with PRN"
```

```
git remote add origin https://github.com/<USERNAME>/Lab_Git_<PRN>.git
```

```
git branch -M main
```

```
git push -u origin main
```

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "Added student name"
```

```
vim Exam_details_<PRN>.txt
```

```
git stash
```

```
git checkout -b feature
```

```
git stash pop
```

```
git add .
```

```
git commit -m "Added welcome message"
```

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "Added current date and time"
```

```
git tag v1.0
```

```
git tag -a release-v1.1 -m "Stable release version 1.1"
```

```
git tag
```

```
git show release-v1.1
```

```
git log
```

```
git log --oneline
```

```
git log --graph --all
```

```
git log --decorate
```

```
git show v1.0
```

```
git show release-v1.1
```

```
git checkout v1.0
```

```
cat Exam_details_<PRN>.txt
```

git checkout feature

git push origin feature

git push origin --tags

git branch

git tag

git stash list

git log --graph --oneline --all

Log out GitHub account from Git Bash (Windows)

First check what credentials Git is using:

```
git config --global user.name
```

```
git config --global user.email
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git config --global user.name
THE7MIST

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git config --global user.email
asombale@gmail.com
```

Remove global identity:

```
git config --global --unset user.name
```

```
git config --global --unset user.email
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git config --global --unset user.name

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git config --global --unset user.email
```

Check stored credentials:

```
git credential-manager erase
```

Or open:

Control Panel → Credential Manager → Windows Credentials

Delete entries such as:

- git:<https://github.com>
- github.com

Verify:

```
git config --global --list
```